Week 5, Clustering

CAPP 30254, Machine Learning for Public Policy

Unsupervised learning

Recall that in supervised learning, labeled examples are used to train machine learning models. In *unsupervised learning*, models learn by unlabeled examples.

Unsupervised learning algorithms can discover structure, patterns, and groupings in data. The most common unsupervised learning methods are dimension reduction and clustering.

supervised learning:

data has labels

$(\underline{x}_i, y_i)$

↑

label

$\{+1, -1\}$

cat, dog

safe, unsafe

Ex.

LR

Logistic regression

unsupervised

no labels

$\underline{x}_i$

Ex.

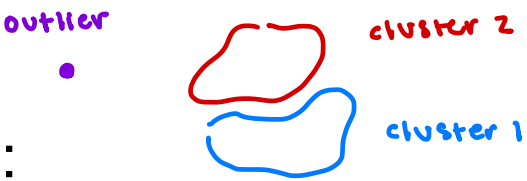
PCA

clustering

Clustering

type of classification, but we learn the classes

Clustering is an unsupervised learning technique that groups data into clusters based on their similarity. The idea is to put similar items in the same cluster and dissimilar items into different clusters.



Clustering can be used to:

not by geography

- Identify areas with similar emergency needs by grouping neighborhoods by type of frequent 911 calls
- Discover patterns in dropout risk by clustering schools based on outcomes

cluster schools then look for similarities

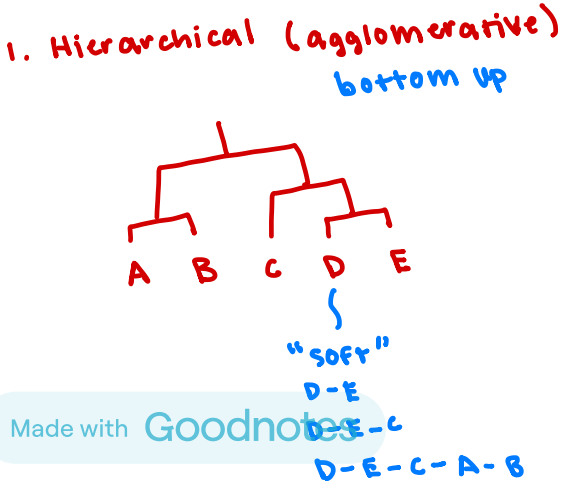
bonus!

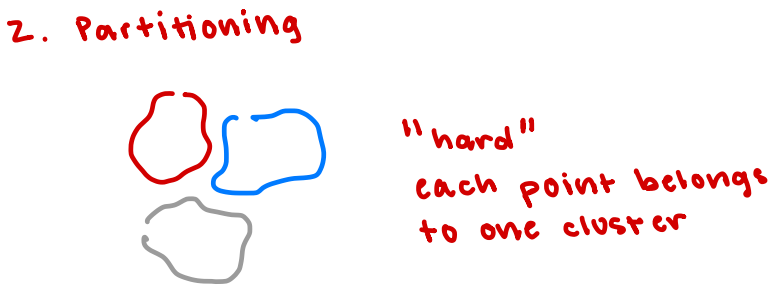
Clustering can also identify anomalies by identifying items that don't fit well into any cluster.

Types of clustering

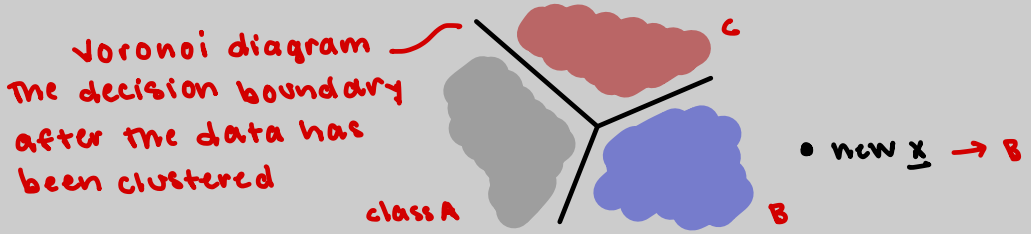
There are a number of approaches to clustering, including:

1. Hierarchical clustering: Build a tree showing how data points group together at different levels of similarity
2. Paritional clustering: Devide the dataset into non-overlapping, flat clusters (e.g., *k*-means)
   - Model-based approaches: Use an assumed underlying probabalistic model to infer the cluster structure (e.g., Gaussian Mixture Models)





k-means clustering



k-means clustering is an algorithm that partitions data into k clusters.

centroid

Consider k clusters where each cluster is represented by its center $\mu_i \in \mathbb{R}^d$. Each datapoint $\mathbf{x}_j \in \mathbb{R}^d$ is assigned to the cluster with the closest center.

$j = 1, \dots, n$
$i = 1, \dots, k$

We want to pick centers that minimize the average square distance:

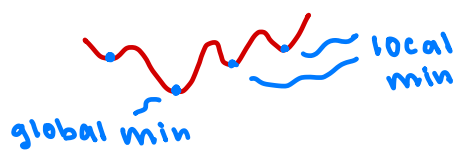
loss: $L(\mu_1, \dots \mu_k) = \sum_{i=1}^{k} \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mu_i\|_2^2$

$\underline{x}$: data point
C_i : cluster i
μ_i : center of cluster i
$\|\underline{x} - \mu_i\|_2^2$: squared distance between $\underline{x}$ and $\underline{\mu}_i$

This is a non-convex minimization problem, so it's difficult to find the optimal solution.

Problem: minimize the loss
argmin $\{\mu_1, \dots, \mu_k\}$
→ find good centers

non-convex:



Lloyd's algorithm

How to find the minimum - even local?

Lloyd's algorithm is an iterative method to solve the k-means clustering problem.

- Initialize the centers: Choose k initial cluster centers

randomly or k-means++

Then repeat until the cluster assignments or centers converge or a maximum number of iterations is reached:

- For each data point $\mathbf{x}_i$, assign it to the nearest center:

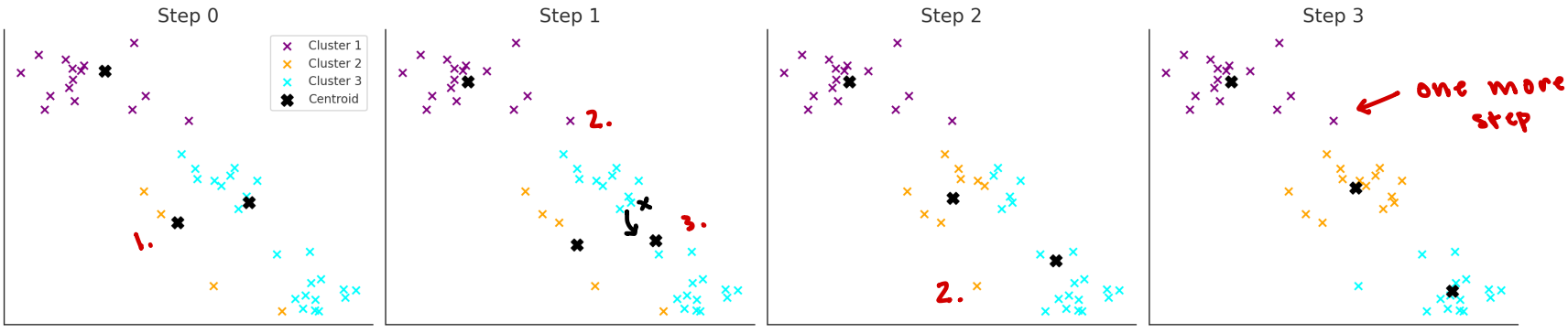
Go through all $\underline{x}$ and assign it to the closest c_i

$$cluster(\mathbf{x}_i) = argmin_j \|\mathbf{x}_i - \mu_j\|_2^2$$

- Recompute each center to be the mean of the data points assigned to that cluster:

$$\mu_j = \frac{1}{|C_j|} \sum_{\mathbf{x} \in C_j} \mathbf{x}$$

Lloyd's algorithm example



- Step 0: Randomly pick 3 centers
- Steps 1-3: Assign points to the closest center and update cluster centers

- clusters become more compact
- centroids shift less as algorithm converges

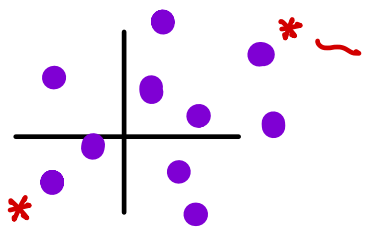
Initializing the centers

How do we pick the centers?

Lloyd's algorithm is not guaranteed to converge to the optimal solution. Its performance depends heavily on the choice of initial centers.

local min in a finite # of steps

local optimum, not global



spread out initial clusters → improves the chance of finding better clusters

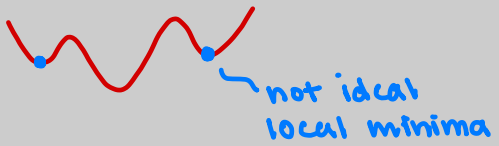
Initialization approaches:

- Random
- Farthest points heuristic
- Multiple random restarts
- Seeding using *k*-means++

next slides

Random restarts

Kind of like cross validation



Multiple random restarts attempt to escape local minima by running the algorithm multiple times, each time starting from different initialization points.

- Pick random centers
- Run Lloyd's algorithm
- Repeat using new random centers
- Pick the best among all runs

Pick the solution that gives the lowest loss function L among the runs

CODE

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3, n_init=10)
```

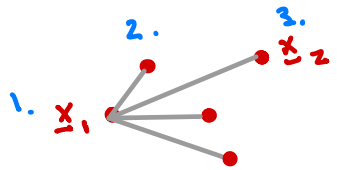
10 random restarts

Seeding *k*-means++

Important to pick good initial centroid values

Regular k-means:
- poor convergence
- unevenly sized clusters
- sensitive to outliers

k-means++ can help avoid poor starting points in *k*-means that lead to non-optimal solutions.



1. First, randomly pick a data point **x** as the first cluster center.
   - 2. Compute the squared distance $d(\mathbf{x}_i, \mathbf{x})^2$ between **x** and all remaining ovservations. Select the next center with the likelihood that it is 3. further away from the points that are already cluster centers.
   - Repeat until *k* centroids have been chosen

k-1 times

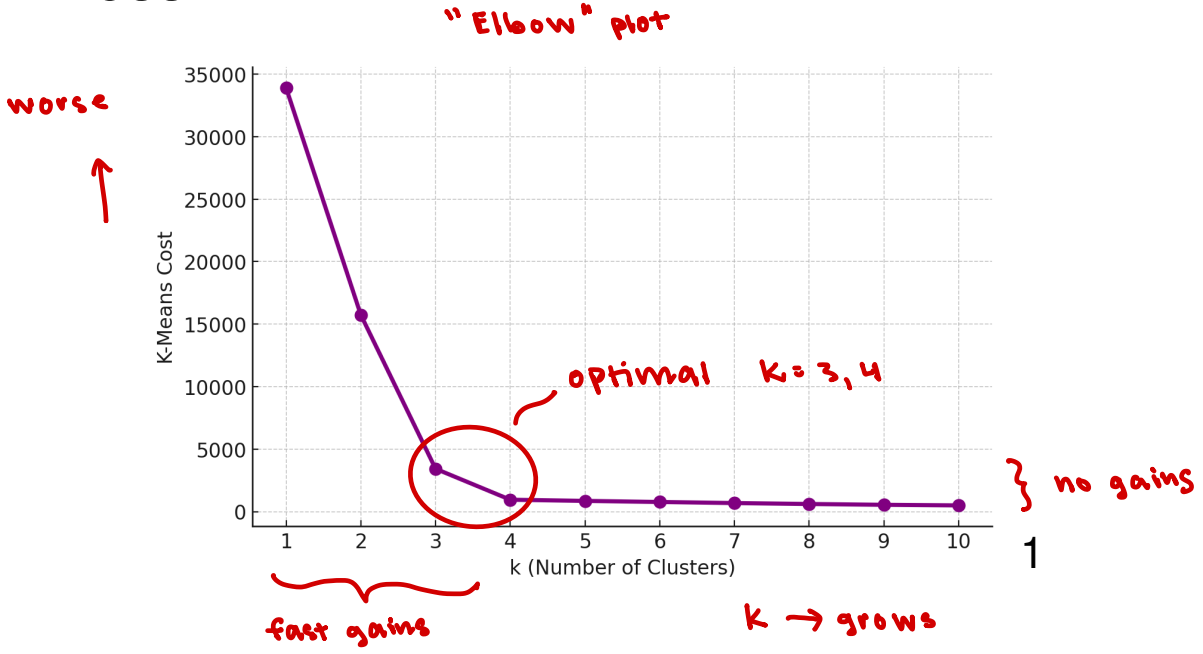
Helps:
- spread out initial centroids
- usually converges faster
- often has a better final clustering

After the center have been chosen, run the standard *k*-means algorithm.

Determining *k*

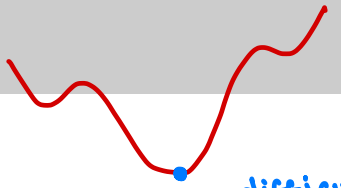
In general, determining the number of clusters *k* is very difficult.

The ”Elbow” plot shows the diminishing returns in the loss function as *k* grows. Pick *k* so that a larger *k* give a negigible decrease in loss.

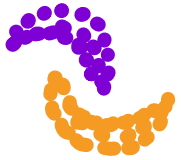


[1] Generated by GPT-4, DALL-E 3 after the prompt: “Can you plot an elbow plot with k = 1 - 10 on the x-asix and the k-means cost function on the y-asix?”

Challenges with k-means



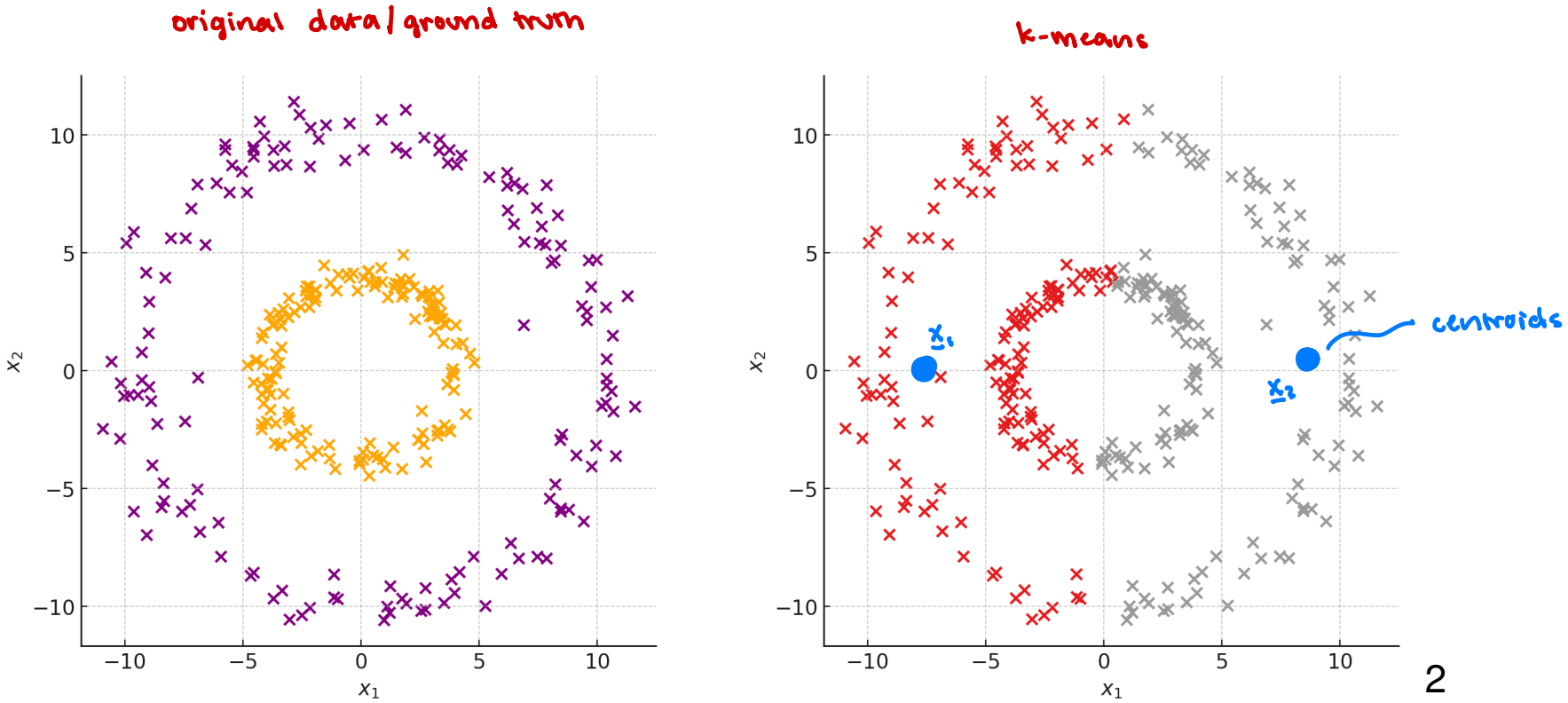
- Usually only converges to local optimum
  - Performance depends on initial choice of centers
  - k-means++ , farthest points, random restarts, but still...
- Number of iterations required can be large sometimes converges quickly in practice
- Cannot model clusters of arbitrary shape
  - Use kernels to separate different shapes



- Determining the number of clusters k is difficult
  - Unsolved problem!

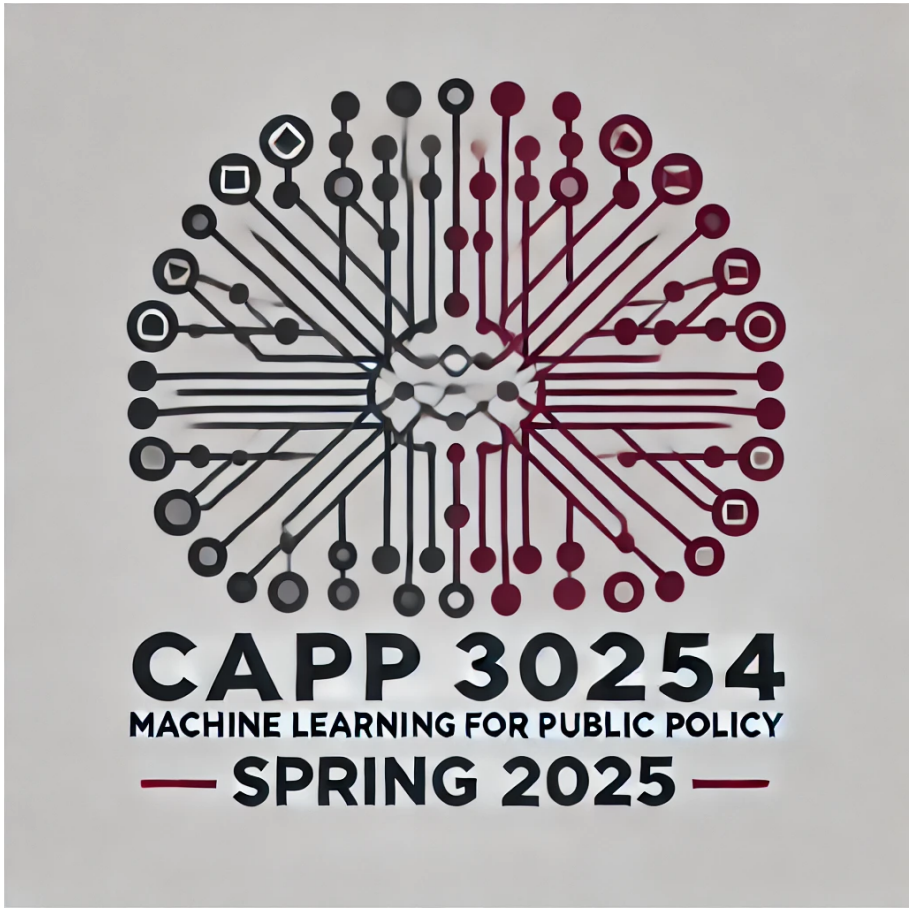
k-means is not always sensible

Some datasets are not ammendable to k-means.



[2]

[2]Generated by GPT-4, DALL-E 3 after the prompt: “Please apply k-means with two clusters to this data where the original starts are on the left and right side of the graph”



[3]

[3] Generated by GPT-4, DALL-E 3 after the prompt: "Hi, could you please make a minimalistic logo for a machine learning class..."